

Negotiated Local Plan Repair for Multi-Robot Warehouse Pathfinding

Sreenitya Thatikunta Arpita Deshmukh Prajna Agrawal
B23CS1072 B23CM1007 B23CS1054

1 The problem

A team of robots works in a warehouse. The floor is a grid of square cells. In each time step, every robot either moves one cell (up, down, left or right) or waits. Each robot has **3 tasks**: go to a shelf, pick up an item, carry it to a delivery station. After its last task it drives back to its own parking spot (its *dock*). Two robots may never be in the same cell at the same time, and may never swap cells with each other in one step.

Before starting, the robots plan collision-free paths. Then, while they follow these paths, **things go wrong**:

- a robot **breaks down** (for 5–20 steps, or for good);
- a cell gets **blocked** (for 10–40 steps, or for good);
- a robot gets an **emergency task** that it must do first, with a deadline.

The robots must then **repair** their plans. The rules of the assignment are: robots fix things by talking to nearby robots; we must not re-plan the whole fleet from scratch; and we should change **as few robots’ plans as possible**.

Our warehouse. The map is 18×33 cells: 25 shelf blocks with one-cell-wide aisles between them, 150 pickup cells next to the shelves, 14 delivery stations on the side walls and 58 docks on the top and bottom walls (Figure 10 on the last page shows it).

Table 1: Key terms used in this report.

Plan	The list of cells a robot will visit, one per time step.
Disruption	A breakdown, a blocked cell or an emergency task that happens while robots move.
Plan repair	Fixing the plans after a disruption, without starting over.
Altered robots	How many robots had to change their plan because of one disruption. Lower is better.
Total time (SoC)	“Sum of costs”: add up the time step at which each robot finishes its tasks. Lower is better.
Extra steps	Total time with disruptions minus total time of the original, undisturbed plan.
Obstacle density	How many cells get blocked during a run, as a share of all free cells (e.g. 5%).
Communication range	Robots can only talk to robots within 6 cells of them.

2 Our approach

Step 1: the initial plan

We use **prioritized planning with Space-Time A***. The robots plan *one after another* (the robot with the longest trip goes first). Space-Time A* is the A* shortest-path search, but on cells *and* time, so it can also decide to wait. Every finished plan is written into a shared **reservation table**, like booking a cell in a calendar: “cell (5,7) is taken by robot 3 at time 12”. Later robots simply plan around the bookings, so no two plans collide.

Step 2: repairing after a disruption

When something goes wrong, only the robots whose plans are now broken are fixed, one at a time. Each one climbs a **ladder of fixes**, from the fix that disturbs nobody else to fixes that involve more robots (Figure 1). It stops at the first fix that is good enough.

1. **Fix it alone (Tier 1).** The robot re-plans its own path and treats everyone else’s plan as fixed. If this costs at most 4 extra steps, accept it. Only one plan changes.
2. **Ask neighbours for help (Tiers 2–3).** The robot finds which nearby robots are in its way and asks a small group of them (1, then 2, then 3 robots) to re-plan together with it. It first asks within 6 cells, then within 12. If a helper is itself blocked, the helper may ask *its* neighbours too (a *chain*, at most 2 levels deep).
3. **Re-plan a small local group (Tier 3b).** Only for urgent cases (an emergency robot, or a robot with no path at all): re-plan the robot together with up to 8 robots near it.
4. **Detour or wait (Tier 4).** Otherwise the robot takes its long detour, or, if it has no path at all, waits where it is and tries again next step. If it has to wait for 10 steps, it starts asking robots further away.

How a robot decides whether asking for help is worth it. Every option gets a score:

$$\text{score} = \text{extra steps for everyone involved} + 3 \text{ for every other robot disturbed} \\ + 10 \text{ if an emergency deadline is missed}$$

Lower is better. A group fix is used only if it scores better than fixing it alone. The “3 per robot” term is how we follow the rule “change as few plans as possible”: disturbing another robot is only worth it if it saves more than 3 steps.

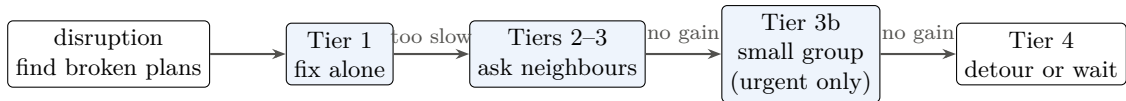


Figure 1: The ladder of fixes. A robot only moves right when the current fix is not good enough, so other robots are disturbed only when it really helps.

What we compare against

- **local: our method** (the ladder above).
- **local-flat:** our first version, without chains, Tier 3b or deadlines.
- **solo:** only Tier 1 (no talking to other robots), to show what negotiation adds.
- **full:** re-plans *every* robot from scratch after every disruption. The assignment forbids this for repair; we use it only as a reference point.

In short: robots plan one after another and book cells in a shared table. After a disruption, each affected robot tries the cheapest fix first and only involves other robots when that saves real time.

3 Experiments and results

How we tested. We ran every setting **10 times** with different random seeds, and gave all four methods exactly the same warehouse and the same disruptions. By default each run has 3% obstacle density, 2 breakdowns and 2 emergencies. We varied the **number of robots** (5 to 50) and the **obstacle density** (0% to 15%). We also ran tests with exactly **one disruption**, to count how many plans

a single disruption changes. In total: 1,700 runs. **No run had a collision**, and every task was completed (except one **solo** run, Section 4).

Table 2: Main comparison at 20 and 50 robots (average of 10 runs, 12 disruptions per run). Read it as: lower total time and fewer altered robots are better.

robots	method	total time	altered per dis.	emergency on time	repair ms	robots	method	total time	altered per dis.	emergency on time	repair ms
20	local	2612	2.06	65%	126	50	local	6958	5.88	35%	2270
	local-flat	2608	2.11	45%	81		local-flat	6974	5.75	15%	1102
	solo	2622	1.93	30%	12		solo	7089	4.62	15%	67
	full	2611	9.22	100%	41		full	7201	39.33	100%	590

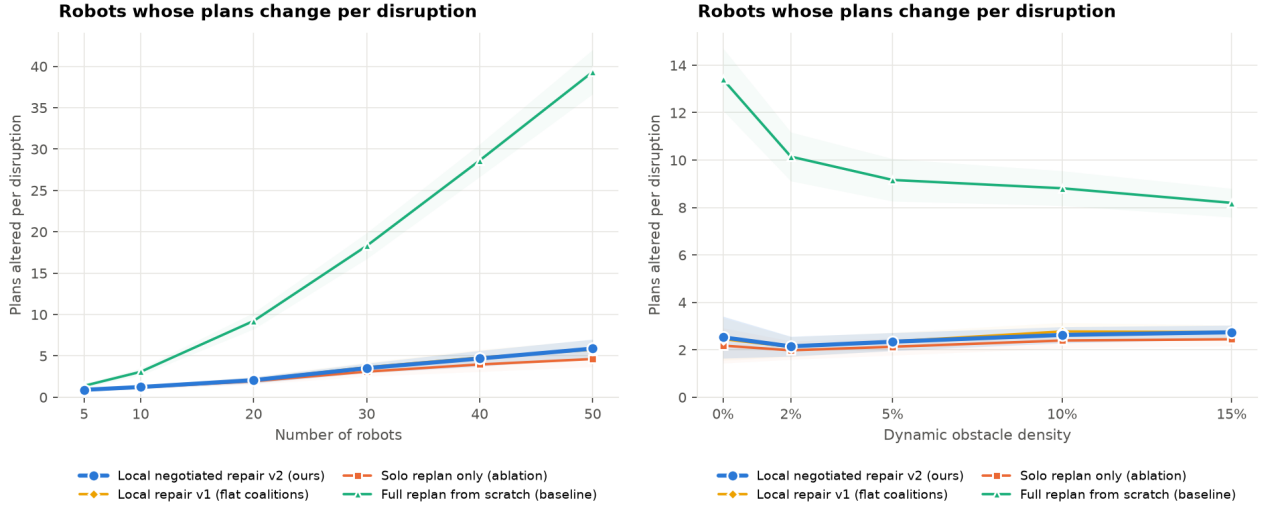


Figure 2: How many robots change their plan per disruption, as the number of robots grows (left) and as obstacles get denser (right). Our method (blue) stays low; re-planning everything (green) changes almost the whole fleet.

Main findings.

1. **We change very few plans.** Our method changes about one robot more than the ones the disruption hits directly. Re-planning everyone changes 9 to 39 robots per disruption.
2. **Re-planning everyone is not even faster.** At 50 robots its total time is *worse* than ours (7201 vs 6958 steps), because it reshuffles all robots, including ones that were never affected.
3. **One disruption, how many plans?** A temporary breakdown changes 2.2 plans (exactly the robots it blocks), a temporary blockage 3.1, and a permanent breakdown 13.5 (the broken robot’s tasks are handed to others). Re-planning everyone changes 22 to 29 (Figure 4, left).
4. **Talking helps more as the fleet grows.** With 50 robots, fixing alone (**solo**) needs 131 more steps than our method.
5. **Improvements over our first version.** Our final version has 7 to 19 fewer total steps at 30 to 50 robots and meets more emergency deadlines (35% vs 15% at 50 robots), while changing about as many plans. The price: repairs take about twice as long.

How performance changes with more robots and denser obstacles

Table 3 lists every setting we tried. Figure 3 shows total time.

Table 3: All results for both experiments (average of 10 runs). Top: number of robots (3% density). Bottom: obstacle density (20 robots; disruptions per run in brackets). “Extra steps” is the time lost to disruptions. “Plans altered” and “repair ms” are per disruption.

	total time			extra steps		plans altered		repair ms	
	local	solo	full	local	full	local	full	local	full
<i>Number of robots</i>									
5 robots	679	681	679	108	109	0.89	1.38	3	5
10 robots	1330	1330	1332	146	148	1.23	3.07	5	10
20 robots	2612	2622	2611	190	189	2.06	9.22	126	41
30 robots	4093	4148	4131	252	290	3.52	18.29	416	124
40 robots	5510	5572	5617	345	452	4.68	28.62	1115	289
50 robots	6958	7089	7201	377	620	5.88	39.33	2270	590
<i>Obstacle density (disruptions per run)</i>									
0% (4)	2565	2568	2555	143	133	2.52	13.40	70	45
2% (9)	2583	2591	2592	161	171	2.14	10.14	61	45
5% (18)	2630	2652	2629	208	208	2.34	9.16	112	62
10% (31)	2768	2831	2813	347	392	2.63	8.81	247	116
15% (45)	2925	2995	2997	503	575	2.74	8.20	320	1712

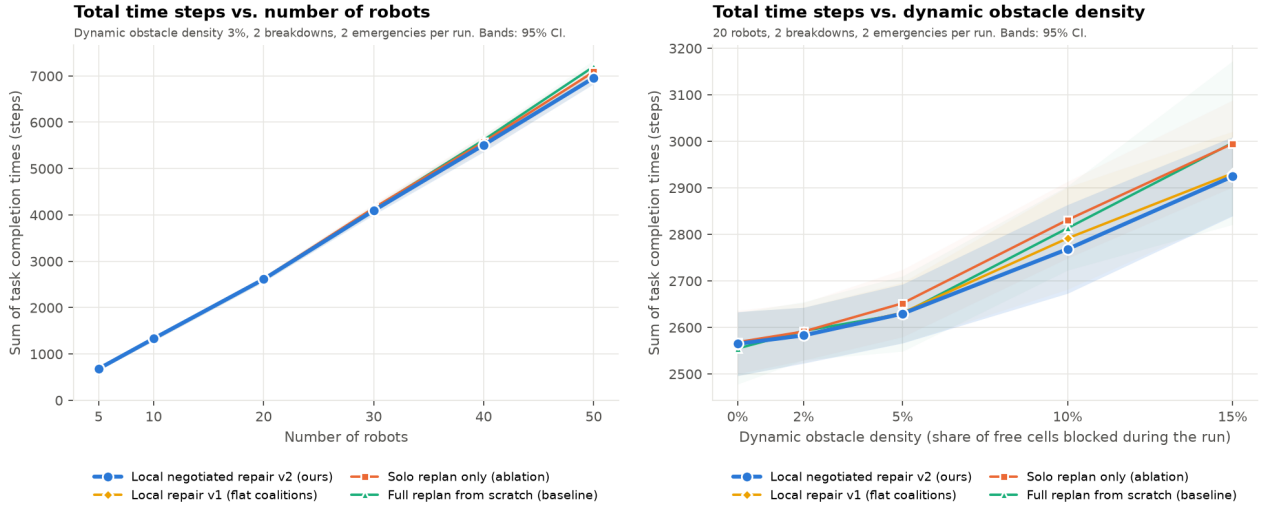


Figure 3: Total time as the number of robots grows (left) and as obstacles get denser (right). Lower is better.

More robots.

- Total time grows in a straight line, about 140 steps per extra robot, because each robot brings 3 more tasks.
- The time *lost* to disruptions grows faster, because a crowded floor has fewer detours: from 108 to 377 extra steps with our method, but up to 620 when re-planning everyone.
- Up to 20 robots all methods are about equal. From 30 robots on, ours is clearly best (37, 107 and 243 steps better than re-planning everyone at 30, 40 and 50 robots).
- Plans changed grow slowly for us (0.9 to 5.9) but almost one-for-one with the fleet when re-planning everyone (1.4 to 39).
- The cost is computing time: our repair takes 3 ms with 5 robots and 2.3 s with 50, because more neighbours means more groups to try.

Denser obstacles.

- Going from 0% to 15% density means 4 to 45 disruptions per run, and our total time rises by 14% (2565 to 2925). Each extra disruption costs about 9 steps.

- Plans changed per disruption stay almost the same (2.1 to 2.7), because a blocked cell only affects the few robots about to cross it.
- At low density all methods are within about 10 steps. From 10% on, ours saves 45 to 72 steps compared with re-planning everyone, and at 15% re-planning everyone becomes slow (1.7 s per disruption vs. our 0.3 s).

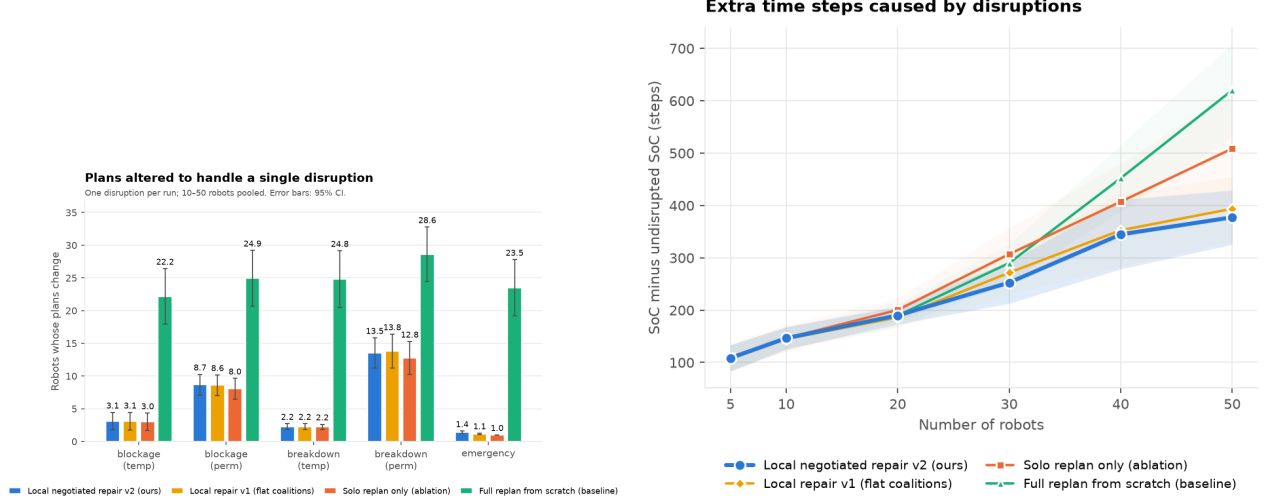


Figure 4: Left: robots whose plans change after a *single* disruption, per type. Right: steps lost to disruptions as the number of robots grows.

The “3 per robot” setting. The value 3 in the score decides the balance between “disturb fewer robots” and “finish faster”. Figure 5 shows what happens for other values (0 to 12): a small value finishes a little faster but disturbs more robots; a large value does the opposite. We chose 3 as a middle ground.

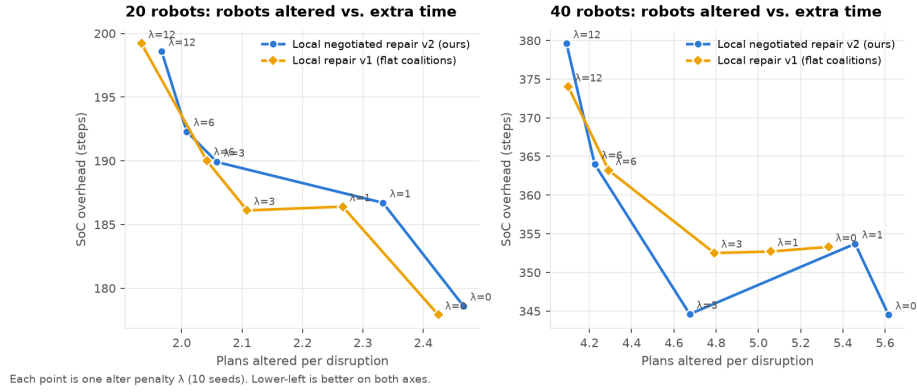


Figure 5: Trade-off between robots disturbed (x-axis) and steps lost (y-axis) for different values of the per-robot penalty λ . Points further down and to the left are better.

In short: our method changes only a few plans per disruption, no matter how many robots or obstacles there are, and it is as fast or faster than re-planning everyone. Its cost is computing time, which grows with the number of robots.

4 When do the robots fail?

In our normal tests the robots almost never fail. One reason is that our test generator never creates impossible situations: for example, it never blocks a cell that would wall off a goal. To find out where things break, we ran a **stress test** (760 runs) in which we turned that safety filter off and pushed the

warehouse harder. A run **fails** if some task is never finished. For each robot that gets stuck we record why:

- **goal cut off**: its goal can no longer be reached at all (the task became impossible);
- **deadlocked**: its goal *can* be reached, but the robot is stuck waiting.

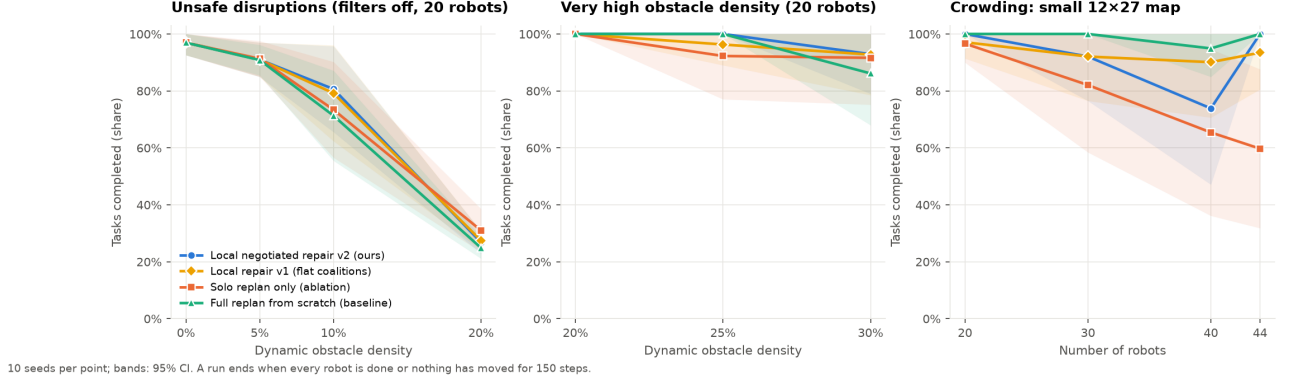


Figure 6: Share of tasks completed in each stress setting. 100% means every task was done.

Table 4: Where the robots fail. “Failed” = runs (out of 10) with an unfinished task. “Stuck” = average stuck robots per run with our method (goal cut off / deadlocked).

setting	value	local (ours)			local-flat	full
		failed	completed	stuck: cut / dead	completed	completed
filter off, half of the events are permanent	density 0%	3/10	96.9%	0.5 / 0.2	96.9%	96.9%
	density 5%	7/10	91.0%	1.5 / 0.3	91.0%	90.8%
	density 10%	10/10	80.6%	2.3 / 3.1	79.2%	71.3%
	density 20%	10/10	26.8%	9.9 / 8.8	27.4%	24.8%
one permanent breakdown, filter off	20 robots	2/10	97.5%	0.4 / 0.2	97.5%	97.5%
	40 robots	4/10	96.7%	0.4 / 1.0	96.7%	96.7%
very dense obstacles, filter on	density 25%	0/10	100%	–	96.3%	100%
	density 30%	1/10	92.9%	0.0 / 2.0	92.7%	86.1%
crowded small map (12 × 27 cells)	30 robots	1/10	92.1%	0.0 / 2.9	92.1%	100%
	40 robots	3/10	73.9%	0.0 / 11.9	90.1%	94.9%
	44 robots	0/10	100%	–	93.4%	100%
short talking range (40 robots)	1, 2, 3, 6 cells	0/10	100%	–	100%	100%

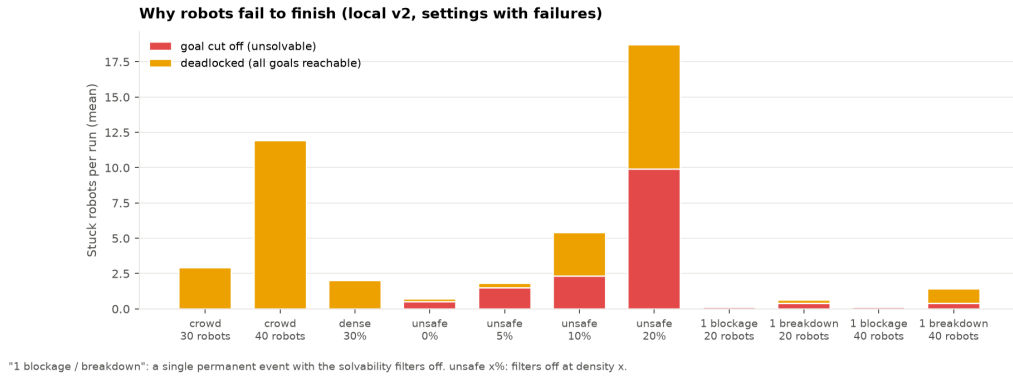


Figure 7: Why robots get stuck with our method. Red: the goal was cut off. Orange: deadlocked, even though the goal could still be reached.

1. Goals get cut off. When a cell is blocked for good, or a robot breaks down for good in a one-cell-wide aisle, it can wall off a pickup cell or a station (Figure 8). Then no method can finish that

task, so all methods fail in the same way. Completion falls from 97% to 27% as the density goes up to 20%. Even a single robot breaking down for good makes 2 to 4 out of 10 runs fail.

2. Stuck robots block others. A robot whose goal is cut off never gives up. It keeps waiting in its cell, and that cell may be the only way through for other robots. At 10% density, 2.3 robots per run have their goal cut off, but another 3.1 get stuck *behind* them. So more than half of the stuck robots are caused by how the robots react, not by the task being impossible. Our method still does better here than re-planning everyone (81% vs 71% completed).

3. Waiting spreads on a crowded map (our main weakness). On a small map with 40 robots, 3 of 10 runs fail with almost every robot stuck. Example: a robot breaks down for just a few steps and blocks 6 robots, which start waiting. Robots that planned to pass through a waiting robot's cell then wait too, and so on, until all 40 robots are waiting for each other. Only 1 of 122 tasks gets done. Our groups are too small (at most 4 or 8 robots) to untangle this, while re-planning everyone solves it. This depends on the exact situation: with 44 robots, no run failed.

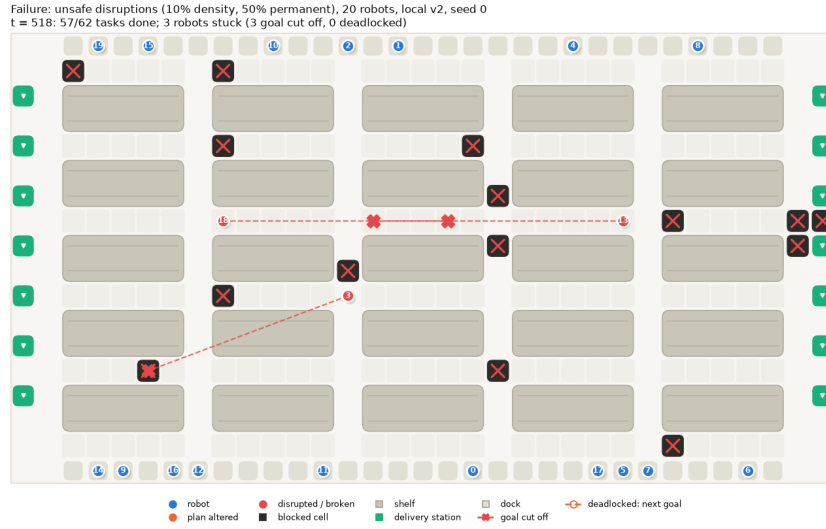


Figure 8: Failure example: permanently blocked cells (black) cut off the goals of robots 3, 13 and 18 (red). 5 of 62 tasks are never done.

4. Other cases.

- **Very dense obstacles:** at 30% density (all tasks still possible), 1 of 10 runs gets a deadlock.
- **No talking:** without negotiation (*solo*), all 49 remaining robots got stuck after one robot broke down for good with 50 robots (Figure 9), and 5 of 10 runs failed on the crowded map. Our method solved both.
- **Emergency deadlines:** in crowded warehouses only 5–35% of emergency deliveries are on time, although all of them are eventually delivered.

Where they did *not* fail: with a very short talking range (even 1 cell, it costs only 1% more time, because a robot that waits too long starts asking robots further away), and in any normal (solvable) test with up to 50 robots and 20% density.

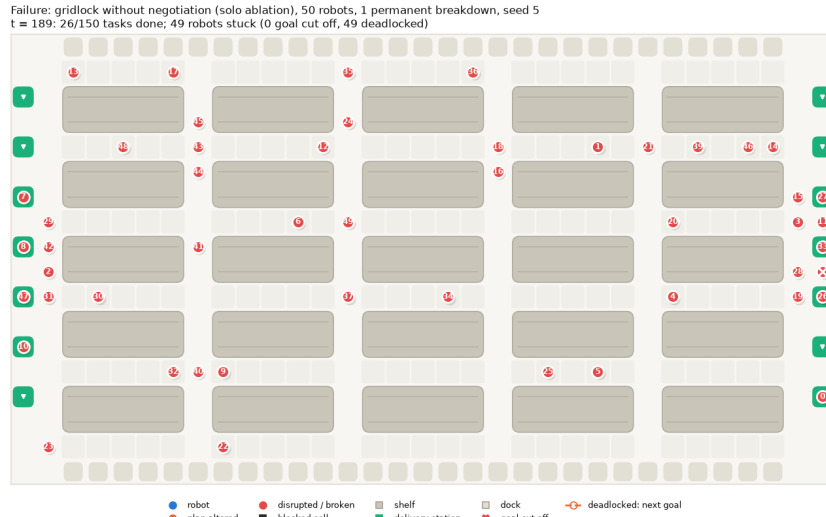


Figure 9: Failure without talking (**solo**): after one permanent breakdown, all 49 remaining robots (red) wait for each other forever. Our method finishes all 150 tasks in the same situation.

In short: robots fail when (1) a goal becomes unreachable, (2) a stuck robot blocks others, (3) waiting spreads through a very crowded map, (4) obstacles are extremely dense, or (5) robots cannot talk to each other.

5 Graphical demonstration

We made the simulations with Python's matplotlib library:

- an **MP4 video** of 20 robots handling 12 disruptions (one frame in Figure 10);
- **before/after pictures** of each type of disruption (old plan dashed, new plan solid);
- **videos of the failure cases** from Section 4;
- an **interactive replay** in the browser and a live replay window.

For a few steps after each disruption, the video colours the robots whose plans changed in orange and draws their new paths.

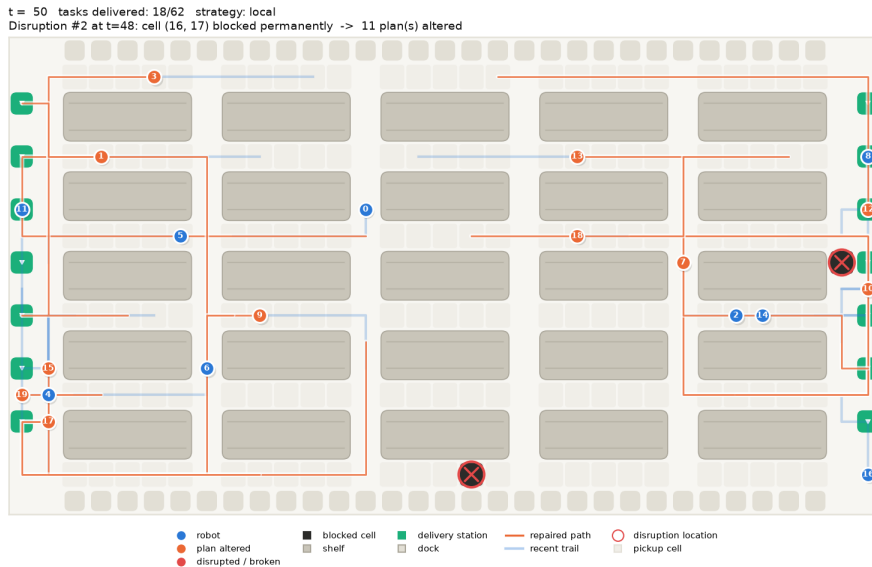


Figure 10: A frame from the video, just after a cell was blocked for good (red circle). 11 robots changed their plans: they are orange, and the orange lines are their new paths.

6 Limitations

- **Planning one robot at a time is not perfect:** it can fail to find a plan even when one exists.
- **Robots never give up a task:** a robot whose goal is cut off waits forever and can block others.
- **Waiting can spread on crowded maps:** a fix would be to re-plan a bigger area when many robots start waiting.
- **Emergency deadlines are soft:** missing one only adds a penalty, so in crowded warehouses most are missed.
- **Repairs get slow with many robots:** about 2 seconds per disruption at 50 robots in Python.
- **Perfect communication is assumed:** messages arrive instantly and are never lost.

Code, results and demos: <https://github.com/SreenityaThatikunta/warehouse-plan-repair>